REPORT For the submission

Basic walkthorugh

There is a normal-expected data preprocessing, mapped the labels to indexes had the labels in sorted manner, created a Customdataset which would take in image directory, label_mapping, transformer and return a tensor and the label

Here is a snapshot of the transforms I used for my dataset, data augmentmentations
are not too heavy as it might distort the image away from its actual emotion.

Val_tranform need not have any data augmentations as we are judging our models on this and if the data transforms and if there is any randomness in the validation set then it would be hard to evaluate models amongst them.

```
train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees = 7),
    transforms.ColorJitter(brightness=0.18, contrast=0.21),
    transforms.RandomAffine(degrees=0,translate=(0.1, 0.1),scale=
(0.9, 1.1)),
    transforms.ToTensor(),
    transforms.RandomErasing(p=0.2, scale=(0.02,0.1))
])

val_transform = transforms.ToTensor()
```

Training Validation data split

I have splitted the data in a 85/15 ratio have random_state to make sure the split remains consist over all the runs, if not implemented the validation data might leak into the model and would be hard to tell if the model has overfitted or not, I have used stratify which makes sure that the ratio of data from labels remains the same and equal to the train set, since the number of sample of disgust is small and the model might not use this class ever in training.

```
from sklearn.model_selection import train_test_split, StratifiedKFo
ld

train_split_2, validation_split = train_test_split(df_label, test_s
ize=0.15,random_state = 37, stratify=df_label["label"])
```

Label	Validation Split	Train Split	Training Set
Happy	0.251219	0.251332	0.251315
Neutral	0.172974	0.172937	0.172942
Sad	0.168331	0.168224	0.168240
Fear	0.142791	0.142693	0.142708
Angry	0.139076	0.139169	0.139155
Surprise	0.110518	0.110442	0.110453
Disgust	0.015092	0.015204	0.015187

Since the classes have such different ratios I implemented the obvious class_weights taking a balanced weights for all of them

Architecture justification:-

After tinkering and the last models being underfit I bumped up the layers to 5 residual blocks

GAP over Flatten+dense:- reduced FC head from 590Kparams to under 2K params

CosineAnnealingLR:- early runs showed F1 oscillating rather than converging cleanly, a decaying schedule was the direct fix, confirmed by later runs.

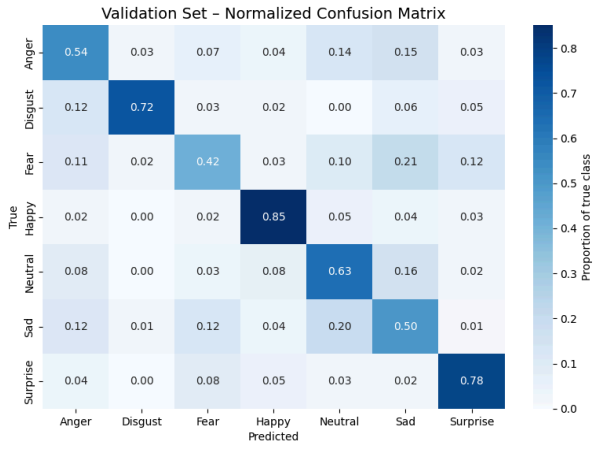
AdamW:- Standard default for CNN trainings

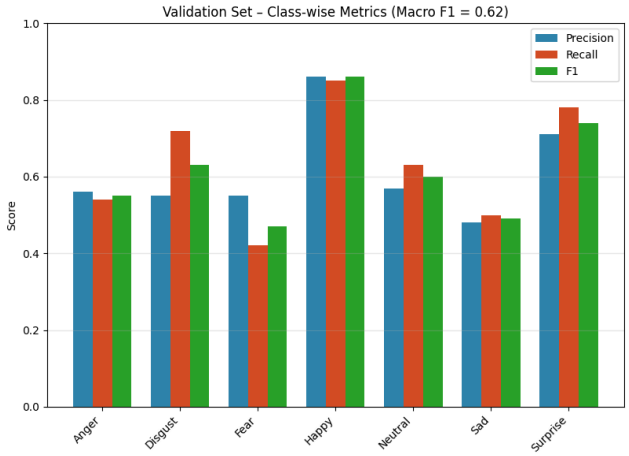
Performance Evidence

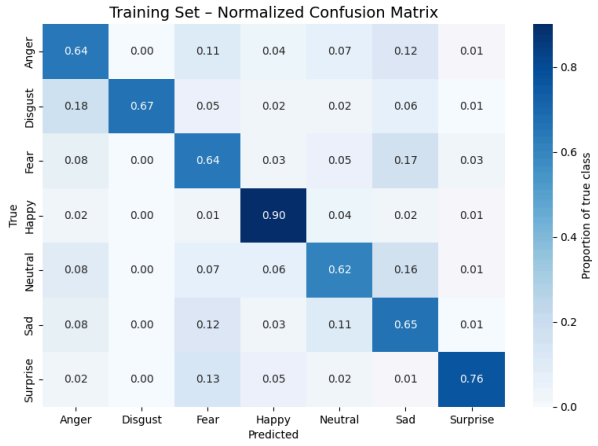
Macro F1 score increased from previous implementations

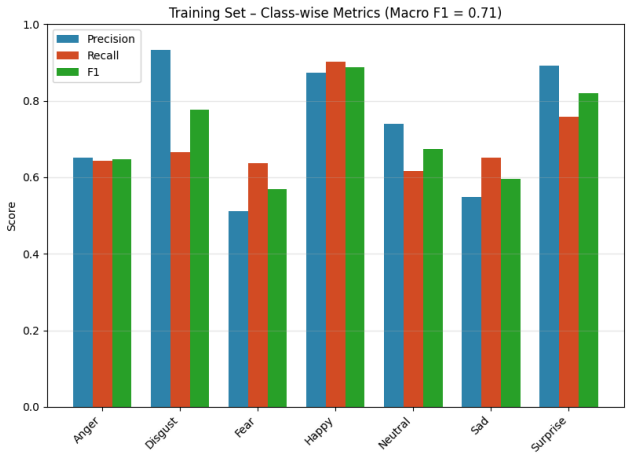
Better precision of Fear so far,

Happy being the strongest class, the model is performing general well for Happy(F1 score = 0.86), Surprise(F1 score = 0.74) and Disgust(F1 score = 0.63)









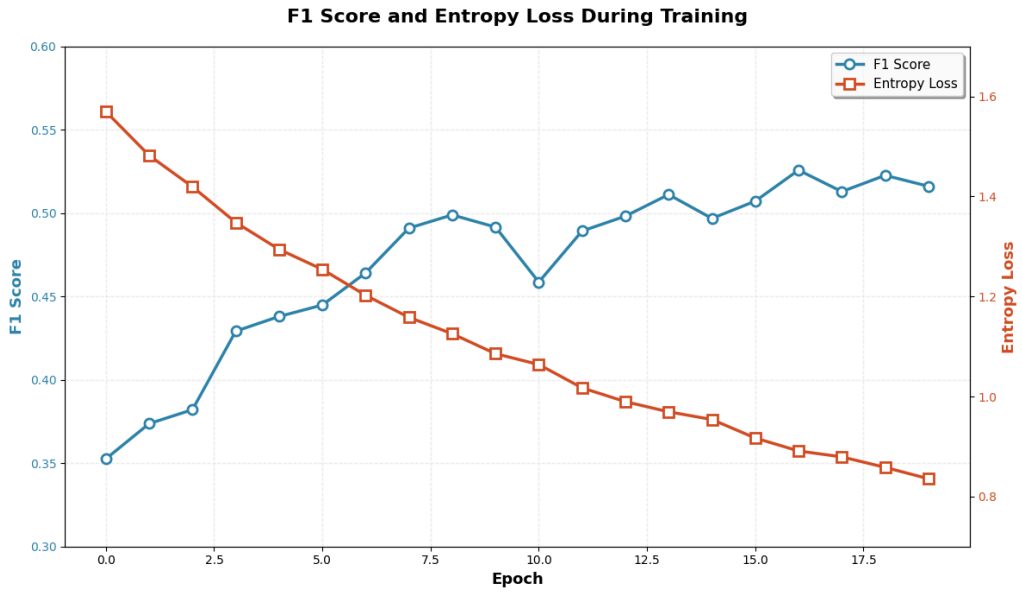
Known limitations:

The model is overfitting ,it is performing better on training set than on validation set f1 score difference being (~0.09) harsher Regularization needs to be implemented

Fear/Sad/Neutral show persistent cross-class confusion, Sad, Neutral are categorised as least expressive emotions, which results to higher error rates.

Version controls:

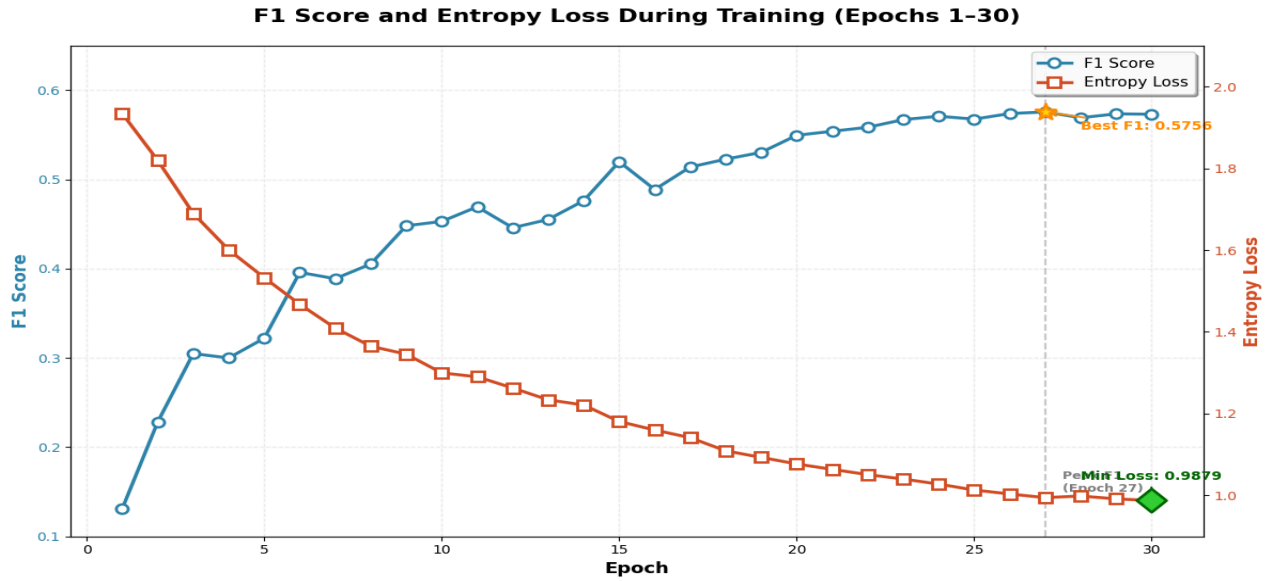
Started with 3 layers on Convo2d each having a ReLU() and a MaxPool(), I chose the CNN to work with as it has proven results and is a standard when it comes to image classification and also Claude told me



This is the very first runthrough on the training set notice that the entropy loss is constantly decreasing but F1score(best: 0.504) has plateaued and the best score was on the 16th epoch, this is before I added the RandomErasing in my transform for my training split
The model is learning, but it has slight-overfitting at the end, and we have to implement some LR decay for this

Version 2-5

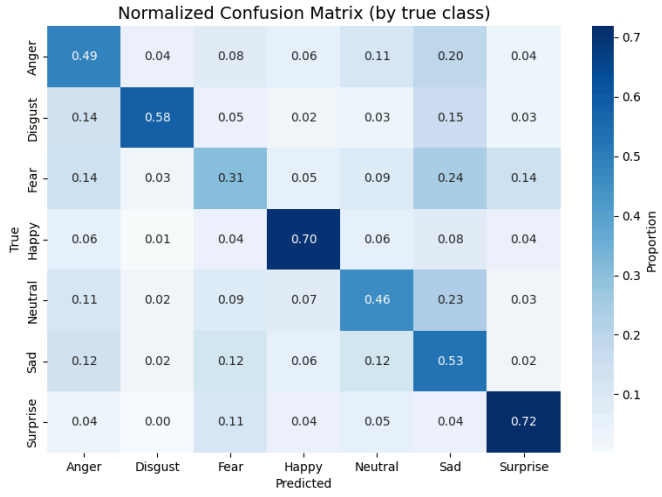
I tinkered around using LR scheduling and added the RandomErasing

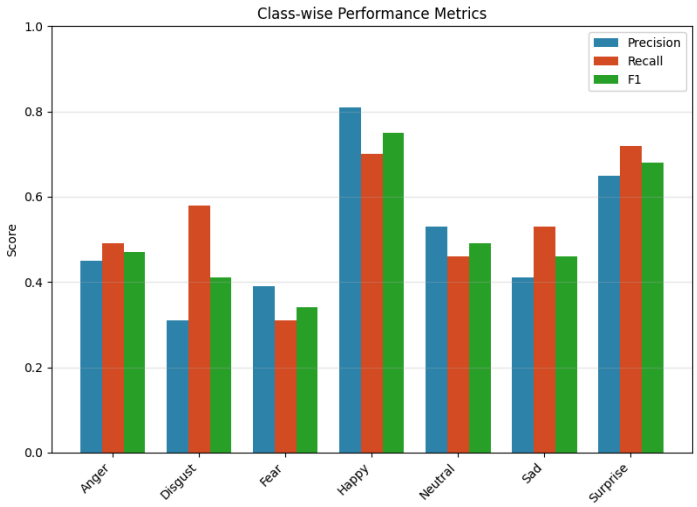


This time the F1 score started at the half of the first version, the curve is much smoother than before and the best is also better than the first version.
This time I also use the confusion matrix and classification report (I only got to know about them, right here)
Happy and surprise are easiest to classify,
Fear being the hardest to classify, model is confusing it with sad, neutral, Surprise

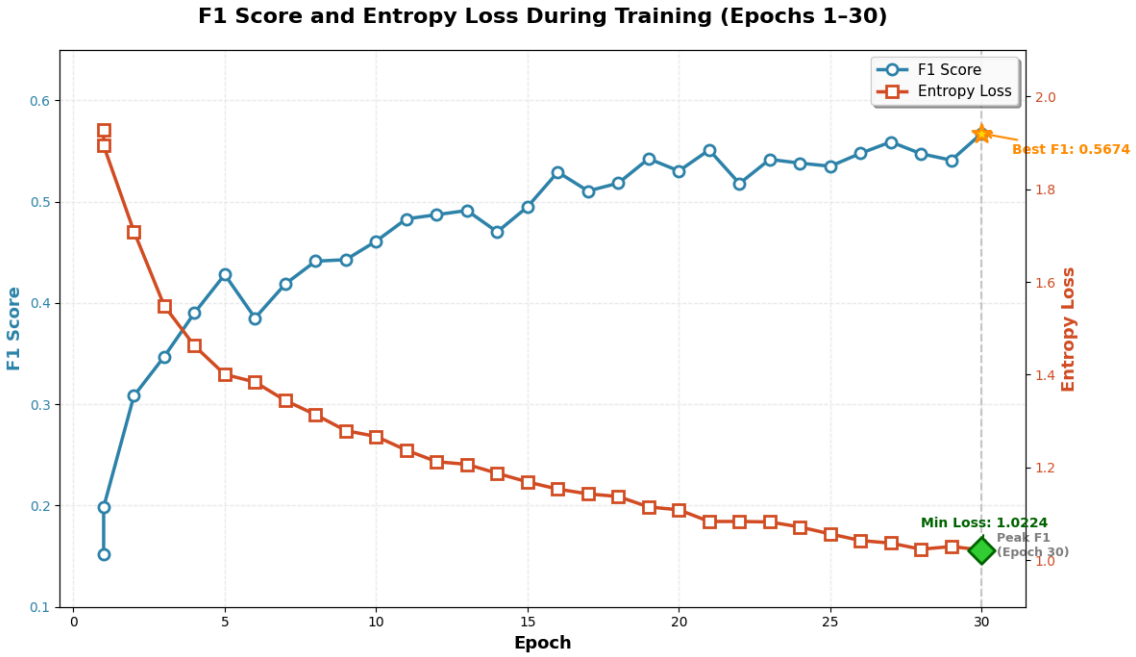
From confusion matrix:-
Anger is often misclassified as Sad, Neutral
Fear is all over the place it is being confused to be anger, sad, neutral, Surprise

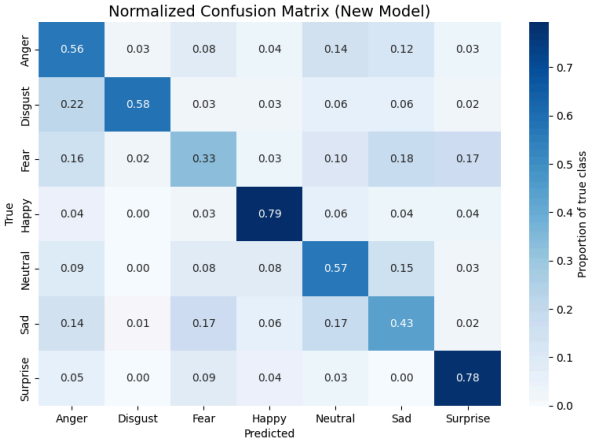
We need to implement a more powerful model for this,

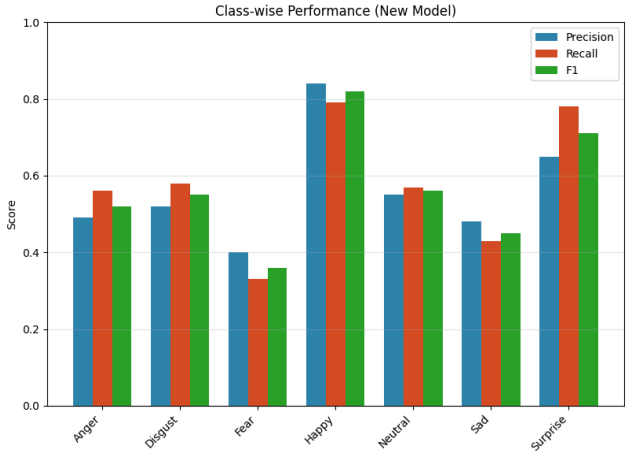




Version 6-7(pun intended) ;)
Inferring from the above (I am bad at inferring) I implemented another layer of CNN with BatchNorm2d 4 layers which then downpooled to 3x3 feature map (this is why the confusion between fear, sad, neutral still persist for this model)







This time around the model got much better at recognising the class but the distinction between the fear sad neutral is still a problem need to be addressed more clearly.

Inferring from this (I warned you I am bad at this) ~~Claude~~ I implemented a 5 residual blocks with GAP replacing Flatten + Linear and only single dropout because of GAP there is much less capacity to regularize against. Data and stats are mentioned above on why I choosed it as my main CNN

```python
class Ultron_level_v3(nn.Module):
    def __init__(self):
        super().__init__()
        self.convo_block = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
        )
        self.faltten_numb = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(0.3),
            nn.Linear(256 * 3 * 3, 256),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(256, 7)
        )
    def forward(self, x):
        y = self.convo_block(x)
        y = self.faltten_numb(y)
        return y
```

KEY TAKEAWAYS:-

Personally instead of tinkering around I should have looked at the confusion matrix and classification report and should have understood why and where is my model lacking, is it overfitting is it because of bidirectional mislabelling, or has the architect reached it max, instead of using different

numbers and hoping it would work and panicking over the f1score, it would have saved time and computations and few hair follicles.

This hackathon was a gateway in the ml world for me, I learned a lot, the ML basics, what is params, working around with data and images , creating transforms and dataset, and the main learning point I have levelled up in **prompt engineering.** It was fun and quickly became frustrating the moment I started focusing too much on increasing the number rather than why to implement this particular method, and looking out for other solutions.

Always save your models, weights, logs, classification reports and confusion matrix, after every run it is very helpful, otherwise at the end it all becomes messy and confusing. Data is the key

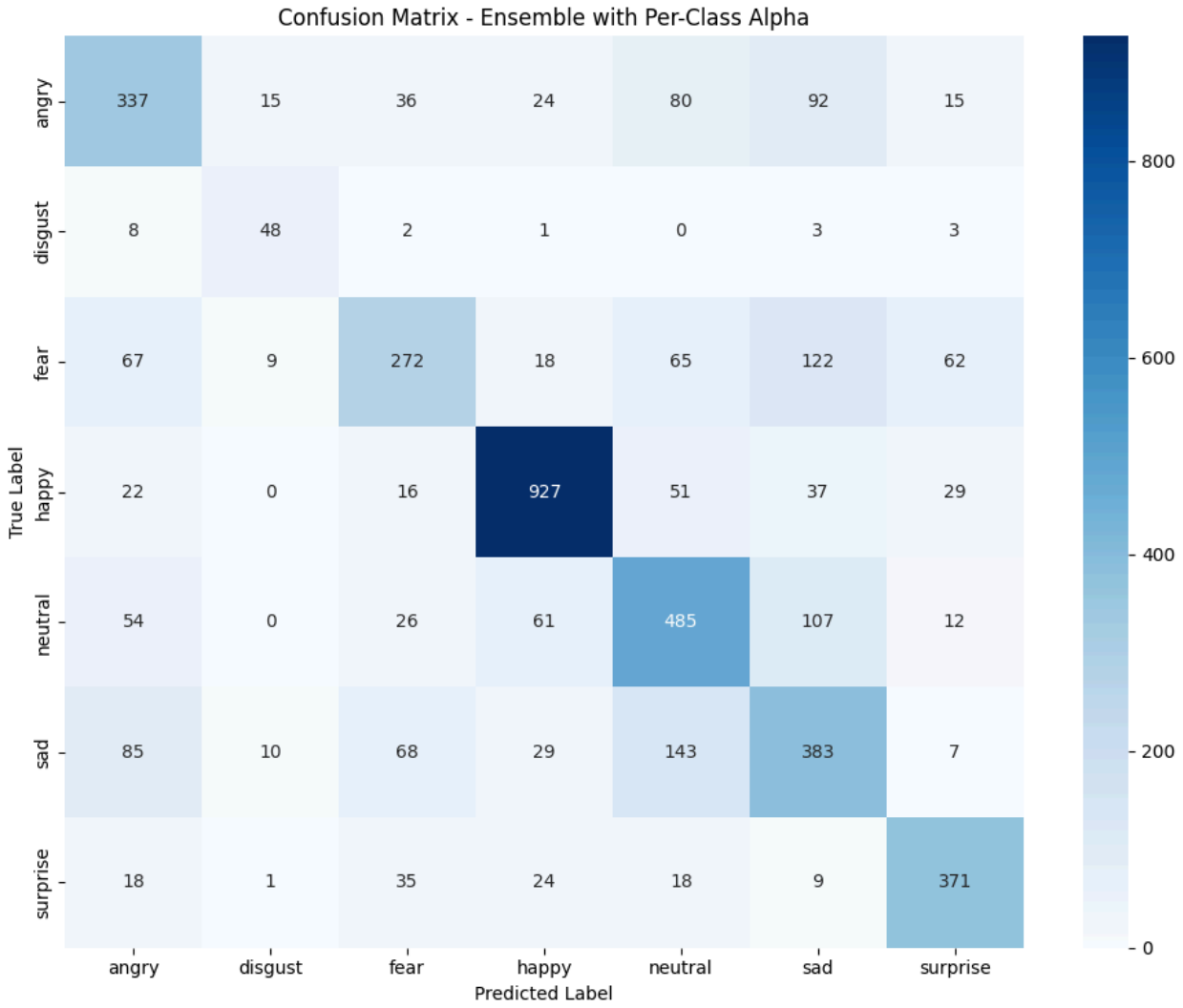
Version control, save your colab notebooks whenever you want to experiment save your previous work with a GOOD description and something like “felt good might delete later”

Have a progress bar which can tell you how much time it will takes and outputs somethings, atleast you can clean your room while you burn a whole lake for performing 0.01 better than before

A SECOND ATTEMPT

I was looking for methods other than CNN, ~~deepseek prompted me~~ I came across XGDBoost XGBoost is an optimized implementation of Gradient Boosting, an ensemble technique that builds a series of decision trees sequentially. Each new tree corrects the errors made by the previous trees, gradually improving predictive accuracy.
I had used the same stratify and the random seed for this one as before so it was fairly easy to compare models.
Why this ?

I already had hit ceiling on my CNN model and nothing seemed to worked,(also for brownie points)
Their Weighted average is performing better than both standalone
I have added the confusion matrix and Final Classification Report



```
FINAL CLASSIFICATION REPORT (Per-Class Optimal Alpha)
================================================================
==============
              precision    recall  f1-score   support

       angry       0.57      0.56      0.57       599
     disgust       0.58      0.74      0.65        65
        fear       0.60      0.44      0.51       615
       happy       0.86      0.86      0.86      1082
     neutral       0.58      0.65      0.61       745
         sad       0.51      0.53      0.52       725
    surprise       0.74      0.78      0.76       476

    accuracy                           0.66      4307
   macro avg       0.63      0.65      0.64      4307
weighted avg       0.66      0.66      0.65      4307
```

Performance against standalone CNN:-

2. Per-Class F1 Breakdown (CNN vs Ensemble vs XGBoost)

Class	CNN F1	XGB F1	Ensemble F1	Δ (Ensemble – CNN)
Angry	0.55	0.394	**0.566**	**+0.016**
Disgust	0.57	0.414	**0.648**	**+0.078** ✅
Fear	0.45	0.401	**0.508**	**+0.058** ✅
Happy	0.85	0.700	**0.856**	+0.006
Neutral	0.60	0.502	**0.611**	+0.011
Sad	0.50	0.401	**0.518**	+0.018
Surprise	0.75	0.663	**0.760**	+0.010

F1 scores:-

CNN (standalone)	64.0%	0.61	0.63
Pixel-XGBoost (first attempt)	43.7%	0.436	0.434
Final Ensemble (per-class α)	66.0%	0.6386	0.65

Where it fails:-
The accuracy is too small
There is a method where makes XGDboost strong (HOG + LBP + geometry +Gabor) but this method is very CPU intensive and slow for the last 1hr and 20min
it is running and still yet to be completed

Why it works:
It is literally wrong + wrong = correct,
The XGDboost perform worse than CNN in every aspects still weight it in increases the accuracy of our model
Despite XGBoost being 10 percentage points worse in accuracy than CNN, it provides diverse errors:
CNN is overconfident and spatially biased (pixel patterns).
XGBoost is underconfident but uses global statistics (histograms/quadrants)
Result: Their errors are decorrelated. When CNN is confidently wrong, XGBoost often has a different opinion. Averaging their probabilities smooths out the noise, lifting performance on the hardest classes (Disgust, Fear).

Improving both individually might not have any change in accuracy, this

works because they both are making different mistakes.

WORD OF HONESTY:- I have used weights, model that I trained from the other notebook in this attempt not pretrained technically,
I have used **Deepseek** to code this whole thing I have no clue what any of the code base for the second part wherever I have implemented the XGDBoost means